

Devoir surveillé

Durée 1h30
(Corrections)

Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
Toute réponse doit être justifiée.

Exercice 1 (Écho es-tu là ? (les signaux) - 7 pts)

Nous souhaitons réaliser une application qui se met en attente de signaux. Dès qu'elle en reçoit un, elle renvoie immédiatement le même signal au processus qui lui a envoyé.

1. Que se passe-t-il si l'application reçoit trop de signaux ? Pourquoi ?

Solution : (1pt) Des signaux seront ignorés car le système ne peut mémoriser tous les signaux, mais utilise un bit pour savoir si un signal a été reçu ou non.

2. Comment un processus peut-il récupérer le PID du processus qui lui a envoyé un signal ?

Solution : (1pt) Lorsque l'on positionne un handler de type `sa_sigaction` (champ de la structure `sigaction`) sur un signal, le paramètre de type `siginfo_t` contient toutes les informations (PID et valeur envoyée).

3. Pourquoi l'utilisation de `sigaction` n'est pas appropriée dans cette application ?

Solution : (1pt) Cela nécessite de faire un appel à `sigaction` pour chaque signal. Ce qui peut être fastidieux.

4. À l'aide de `sigwaitinfo`, donnez le code de l'application (sans les includes et sans la gestion d'erreur).

Solution : (2pts) Une solution :

```
sigset_t sigs;
siginfo_t info;
int num_sig;

/* Blocage de tous les signaux */
sigfillset(&sigs);
sigprocmask(SIG_BLOCK, &sigs, NULL);

/* Boucle principale */
while(1) {
    /* Attente du signal */
    printf("En attente d'un signal (PID=%d)\n", getpid());
    sigfillset(&sigs);
    if((num_sig = sigwaitinfo(&sigs, &info)) == -1) {
        perror("Erreur sur sigwaitinfo");
        exit(EXIT_FAILURE);
    }
    printf("Signal reçu : num=%d / PID=%d / val=%d\n", num_sig, info.si_pid,
        info.si_value.sival_int);

    if(sigqueue(info.si_pid, num_sig, info.si_value) == -1) {
        perror("Erreur lors de l'envoi du signal");
        exit(EXIT_FAILURE);
    }
}

return EXIT_SUCCESS;
```

5. Que se passe-t-il si cette application reçoit le signal SIGKILL ?

Solution : (1pt) Elle est interrompue sans envoyer de signal car ce signal ne peut être bloqué.

Nous souhaitons maintenant pouvoir envoyer en même temps que le signal, un message de type `char*`.

6. Est-ce possible ? Si oui, expliquez comment le mettre en œuvre. Sinon, expliquez pourquoi.

Solution : (1 pt) Il est possible d'envoyer des données à l'aide de `sigqueue`. Cependant, la donnée doit être soit un entier, soit un pointeur. Dans le deuxième cas, il faut que le pointeur soit dans le même espace d'adressage. Ce qui n'est pas possible ici.

Exercice 2 (Disque dur virtuel - 9 pts)

Nous souhaitons réaliser une application de disque dur virtuel dans un fichier binaire. Un programme permet de copier ou de récupérer des fichiers "réels" dans/depuis le disque virtuel. Pour cela, nous utilisons une structure (nommée `index`) située au début du fichier permettant de représenter les entrées (les fichiers stockés dans le disque dur virtuel). Chaque entrée est identifiée par un nom de 16 caractères maximum et une position dans le fichier.

1. Pourquoi est-il préférable que l'index possède un nombre d'entrées fixe ?

Solution : (1 pt) Si le nombre n'est pas fixe, la taille de l'index va devoir augmenter à chaque ajout de fichier. Il faudrait alors déplacer le fichier suivant à un autre endroit. Cela entraînerait également des "trous" dans le fichier.

2. Si le nombre d'entrées est trop grand, nous souhaitons pouvoir ajouter un autre index dans le disque dur virtuel. Expliquez comment procéder.

Solution : (1 pt) On ajoute un champ `suivant` indiquant la position de la prochaine table. Si le nombre d'entrées est trop grand, on ajoute une nouvelle table à la fin du fichier. La position de la nouvelle table est indiquée dans le champ `suivant` de la table précédente.

3. Comment représenter les entrées vides ?

Solution : (1 pt) On peut utiliser un nom de fichier particulier (vide, par exemple) ou une position à 0 (ce qui n'est pas possible car cela correspond à la position du premier index).

4. Proposez une structure en C pour l'index (vous pouvez utiliser d'autres structures si nécessaire).

Solution : (1 pt) Voici une solution :

```
#define MAX_ENTREES 16
typedef struct {
    off_t position; /* Position dans le fichier */
    char[16] nom; /* Nom de l'entrée */
    off_t taille; /* Taille du fichier */
} entree_t;
typedef struct {
    entree_t entrees[MAX_ENTREES]; /* Les entrées */
    off_t suivant; /* La position de l'index suivant */
} index_t;
```

5. Écrivez la fonction `int creer_disque(char *nom_disque)` qui permet de créer un nouveau disque virtuel dont le nom est passé en paramètre et qui retourne le descripteur de fichier correspondant.

Solution : (2 pt) Voici une solution :

```
int creer_disque(char *nom_fichier) {
    index_t table;
    int i;
    int fd = open(nom_fichier, O_CREAT|O_WRONLY, S_IRUSR|S_IWUSR);

    for(i = 0; i < MAX_ENTREES; i++) {
        table.entrees[i].position = 0;
        sprintf(table.entrees[i].nom, "%0");
    }
    table.suivant = 0;
```

```

    write(fd, &table, sizeof(table));

    return fd;
}

```

6. Donnez le code de la fonction `off_t rechercher(int fd, char *nom_fichier)` permettant de retourner la position d'un fichier dans le disque virtuel (ou 0 si non trouvé). Le paramètre `fd` est le descripteur de fichier correspondant au disque virtuel (on ne connaît pas la position actuelle du curseur) et `nom_fichier` est le nom du fichier recherché.

Solution : (2 pt) Voici une solution :

```

off_t rechercher(int fd, char *nom_fichier) {
    index_t table;
    int i, stop = 0;
    off_t pos = 0;

    lseek(fd, 0L, SEEK_SET);
    read(fd, &table, sizeof(table));

    while((stop == 0) && (pos == 0)) {
        i = 0;
        while((i < MAX_ENTREES) && (strcmp(table.entrees[i].nom, nom_fichier) != 0)) i++;
        if(i == MAX_ENTREES) {
            if(table.suivant != 0) {
                lseek(fd, table.suivant, SEEK_SET);
                read(fd, &table, sizeof(table));
            }
            else
                stop = 1;
        }
        else
            pos = table.entrees[i].position;
    }

    return pos;
}

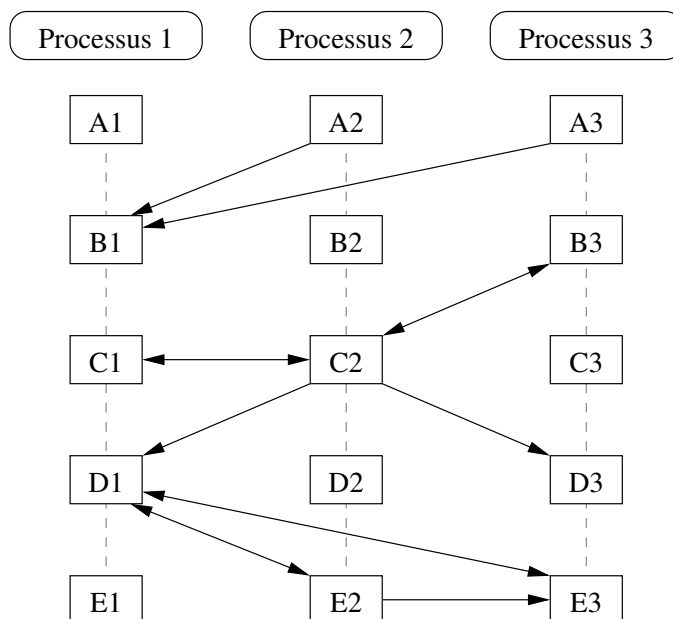
```

7. Expliquez comment ajouter un nouveau fichier dans le disque virtuel (détaillez bien les appels système nécessaires).

Solution : (2 pt) On se place au début du disque virtuel (`lseek`) et on lit l'index (`read`). On parcourt les entrées de l'index et on cherche une entrée vide. Si toutes les entrées sont occupées, on passe à l'index suivant (`lseek + read`) et ainsi de suite. Si on ne trouve pas d'entrée vide, on va à la fin du fichier (`seek`), en sauvegardant la position, et on stocke un index vide (`write`). On revient sur le dernier index (`lseek`) et on modifie la position de l'index suivant (`write`). On peut alors retourner sur le nouvel index : la première entrée est forcément vide. Une fois l'entrée vide trouvée (ou créée), on va à la fin du fichier (`lseek`) et on conserve la nouvelle position. On copie le fichier dans le disque virtuel à l'aide de `read` et `write` (lecture/écriture par blocs). On revient sur la position du dernier index lu. On modifie l'entrée que l'on avait trouvée (en spécifiant le nom du fichier, sa position et sa taille) et on sauvegarde la modification dans le fichier (`write`).

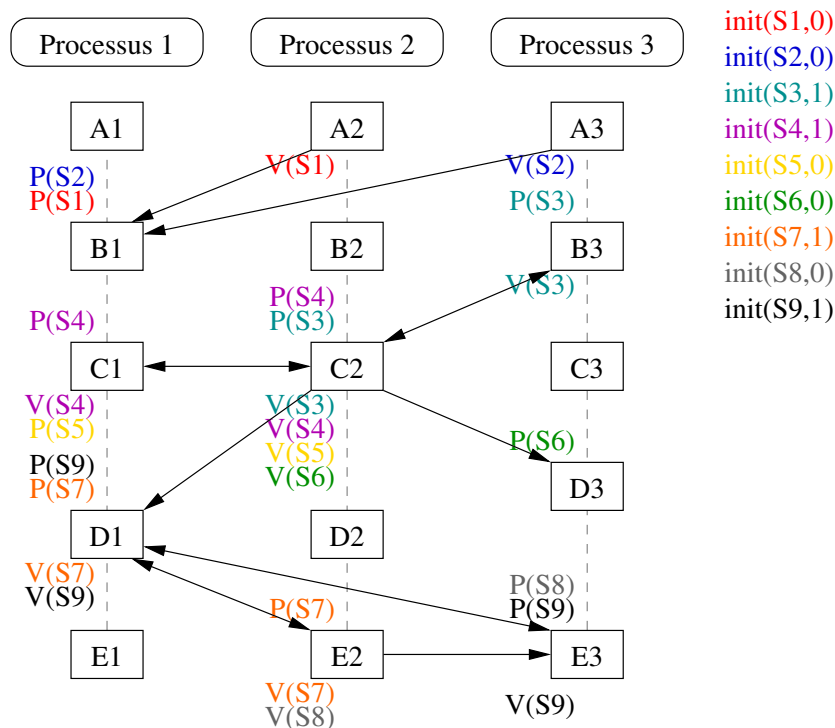
Exercice 3 (Sémaphores de Dijkstra - 4 pts)

Soit le diagramme de précédence suivant, représentant 3 processus contenant chacun 5 blocs s'exécutant les uns après les autres :



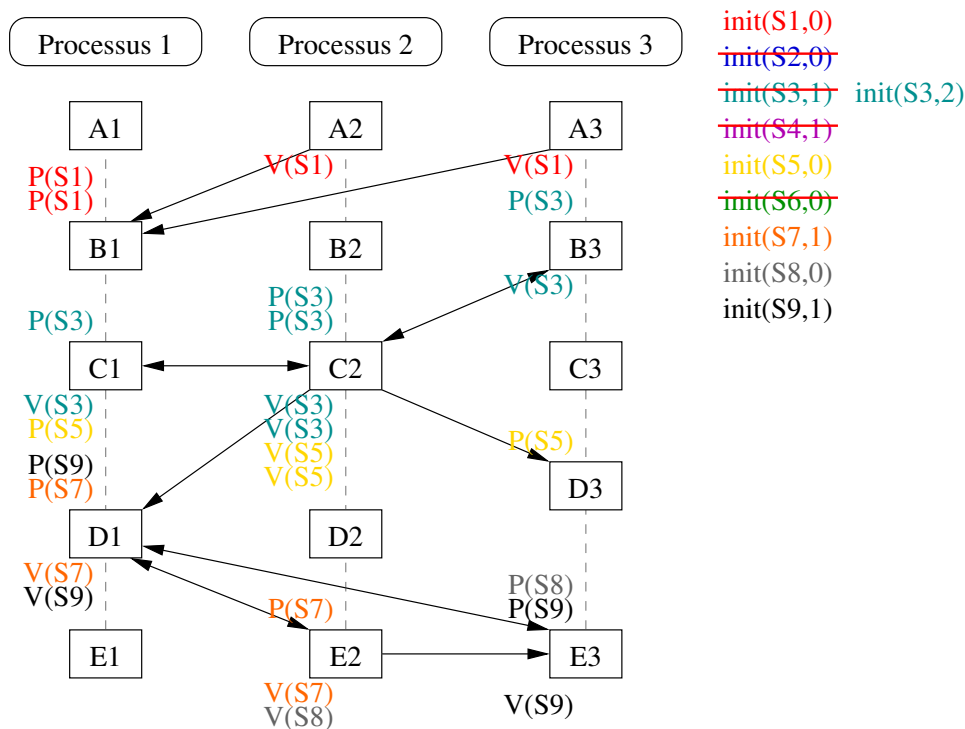
1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

Solution : (2pts) Voici une solution :



2°) Proposez une solution pour réduire le nombre de sémaphores.

Solution : (1pts) Les sémaphores 2 et 6 peuvent être supprimés, ainsi que le sémaphore 4 si le sémaphore 3 est initialisé à 2.



3°) Expliquez pourquoi la solution proposée pour résoudre les contraintes entre C1, C2 et B3 n'est pas satisfaisante avec les sémaphores de Dijkstra.

Solution : (1pts) Supposons que le processus 3 désire entrer en premier en exclusion mutuelle sur B3. Il réalise $P(S3)$ en premier puis commence à exécuter son code B3. Le processus 2 souhaite également entrer en section critique sur C2. Il exécute $P(S4)$ et reste bloqué sur $P(S3)$. Puis le processus 1 souhaite entrer en section critique sur C1. Il est bloqué sur $P(S4)$ alors que le bloc C2 n'est pas en cours d'exécution. Cela vient du fait qu'il n'est pas possible de réaliser de manière atomique les deux opérations $P(S4)$ et $P(S3)$.